SIMATS ENGINEERING

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS
ASSIGNMENT 1 – CO1 Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System

---

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	**Details**	**Field**	**Details**
Student Name	Ch. Tharunika Reddy	Register Number	192324307
Department / Section	AIDS	Academic Year / Semester	4th Year
Faculty Name	DR. M. D. Boomija	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	05-11-2026	Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	**Title**	**Max**	**Awarded**	**Evidence / evaluation focus**
1	Introduce Cloud Computing	3		Definition; characteristics; deployment models; scenario mapping
2	Trace Evolution and Hardware Evolution	3		Timeline; hardware evolution; relevance to university
3	Analyse Internet Software Evolution	3		Web/software evolution; SOA; APIs; scenario relevance
4	Apply Server Virtualization	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Apply Web Services	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Analyse IaaS, PaaS and SaaS	5		Responsibility layers; fiveapplication mapping; comparison; providers
7	Examine XaaS	3		Four or more variants; two justified choices; provider risks
8	Design Cloud Solution Architecture	5		Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Documentation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots

10	Evaluate and Recommend Final Solution	7		Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8		Impact; evaluation; adaptation; evidence-based defence
	TOTAL	**50**		**Percentage :**

OVERALL REMARKS

_____ **Student Signature**	_____ **Faculty Signature**	_____ **Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Cloud definition, characteristics, deployment models and scenario mapping		All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance		Well explained with relevant examples
3	Internet software evolution, SOA and APIs		Clear explanation and scenario relevance
4	Hypervisors, VM/container comparison and consolidation calculations		Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs		API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping		All five applications categorized with justification
7	XaaS variants and two justified selections		Six variants explained; DBaaS and DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery		Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots		Code and procedures provided; actual execution evidence requires verification
10	Three-year cost comparison, risks and migration roadmap		Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation		Changed requirements evaluated; loadtest evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code		Python Flask REST API and Dockerfile documented
Virtualization		Docker build and run procedures provided; execution screenshot required
PaaS or SaaS		Cloud deployment steps described; actual deployment confirmation required
Functional test		HTTP 200 and expected JSON response documented; actual test output required
Failure case		HTTP 404 test case included; execution evidence required
Screenshots		Required screenshots identified but authenticity and completeness not verified
Reproducibility		Installation, configuration, commands and testing steps documented

Interpretation		Technical outcomes and implementation limitations explained
Code and report consistency		Proposed API, Docker configuration and testing procedure are aligned

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	Status
All 11 tasks answered	☐ Yes ☐ Partly ☐ No
Cloud concepts and comparisons explained	☐ Yes ☐ Partly ☐ No
Virtualization calculations included	☐ Yes ☐ Partly ☐ No
API examples and service models explained	☐ Yes ☐ Partly ☐ No
Cloud architecture diagram included	☐ Yes ☐ Partly ☐ No
Implementation and testing evidence provided	☐ Yes ☐ Partly ☐ No
Three-year cost analysis and roadmap included	☐ Yes ☐ Partly ☐ No
Day-4 design challenge completed	☐ Yes ☐ Partly ☐ No
Figures and screenshots included	☐ Yes ☐ Partly ☐ No
Table and figure numbering correct	☐ Yes ☐ Partly ☐ No
Viva recommended	☐ Yes ☐ No

CSA15 – Cloud Computing and Big Data Analytics

ASSIGNMENT 1

CO1: *Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.*

Assignment Title: Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System (Total Marks – 50)

Duration: 1 week

Scenario – Smart Campus Cloud Migration

A multi-campus university currently runs its Student Information System, Learning Management System, Library System, Exam Portal and Email on ageing physical servers in a single on-premise data centre. Each application has its own dedicated server, which stays idle for most of the year but fails to cope during admissions, result publication and onlineexam weeks.

The university's current environment is:

- 12 physical servers, average CPU utilisation of about 15%, with peaks above 95% during admissions and exams
- Approximately 25,000 students and 2,000 staff across 4 campuses
- Applications built as large monolithic programs, with no public interfaces for mobile apps or partner colleges
- High capital expenditure, slow procurement (6–8 weeks per server) and no disaster recovery site

As a cloud consultant, you must study how computing evolved to the cloud model, and then design a cloud-based solution that uses virtualization, web services and an appropriate mix of service models (IaaS, PaaS, SaaS and XaaS) for the university. All recommendations must be justified in terms of cost, scalability, availability and manageability.

Design Challenge:

On Day 4, the university announces that it will launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10 times the current peak, with a requirement of 99.9% availability and a limited operating budget. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Introduce Cloud Computing and Map It to the Scenario (3 Marks)

Design and Propose cloud computing model for the context of the university's problems.

- Definition of cloud computing and the five essential characteristics (on-demand selfservice, broad network access, resource pooling, rapid elasticity, measured service)
- Deployment models (public, private, hybrid, community) and which one suits the university
- Three problems of the current on-premise setup and how each cloud characteristic addresses them
- Benefits and limitations of cloud adoption for the university

Evidence Expected

A clear explanation of cloud characteristics and deployment models, mapped to the university's actual problems.

Answer:

TASK 1 – Introduce Cloud Computing and Map It to the Scenario

1.1 Introduction to Cloud Computing

Cloud computing is a technology that provides computing resources such as servers, storage, databases, networking and software through the internet. Instead of purchasing and maintaining physical servers, organizations can access resources whenever required and pay according to their usage.

For the university, cloud computing provides a centralized platform to manage student records, examinations, library services, learning management and email across four campuses.

1.2 Five Essential Characteristics of Cloud Computing

1. On-Demand Self-Service: Users can access computing resources whenever needed without contacting the service provider. The university can create additional servers during admission periods.

2. Broad Network Access: Cloud services can be accessed through laptops, smartphones and tablets using the internet. Students and faculty can access university applications from all four campuses.

3. Resource Pooling: Computing resources are shared among multiple users through a common infrastructure. The university can use shared cloud resources instead of maintaining 12 separate physical servers.

4. Rapid Elasticity: Resources can automatically increase or decrease according to demand. During examination results and admissions, additional computing capacity can be provided.

5. Measured Service: Cloud usage is monitored and measured. The university pays based on the resources consumed, reducing unnecessary infrastructure expenses.

1.3 Cloud Deployment Models

Model	Description	University Application
Public Cloud	Services are provided through shared infrastructure by a cloud provider.	Hosting student portals and web applications.
Private Cloud	Cloud infrastructure is dedicated to one organization.	Managing highly sensitive student information.

Hybrid Cloud	Combination of public and private cloud.	Keeping sensitive records private and hosting public services on public cloud.
Community Cloud	Infrastructure shared by organizations with common requirements.	Sharing educational resources among affiliated colleges.

Recommended Model: Hybrid Cloud

The hybrid cloud model is suitable for the university because it combines the advantages of public and private cloud environments.

- Sensitive student records and financial information can be maintained in a controlled private environment.
- Public cloud resources can host the LMS, student portal and library search services.
- Resources can be increased during admission and examination periods.
- It provides better flexibility, security and cost management.

1.4 Problems in the Existing System and Cloud Solutions

Existing Problem	**Cloud Characteristic**	**Proposed Solution**
12 physical servers remain idle with only 15% average CPU usage.	Resource Pooling	Combine resources using virtualization.
Servers cannot handle admission and examination peaks.	Rapid Elasticity	Automatically increase server capacity.
Server procurement takes 6–8 weeks.	On-Demand Self-Service	Provision cloud resources within minutes.

1.5 Benefits of Cloud Adoption

1. Cost Reduction: Reduces the need to purchase and maintain physical servers.
2. Scalability: Resources can be increased during peak workloads.
3. High Availability: Cloud providers offer backup and multiple availability zones.
4. Accessibility: Students and faculty can access services from different campuses.
5. Disaster Recovery: Data can be backed up in another geographical region.
6. Easy Management: Centralized infrastructure simplifies administration.

1.6 Limitations of Cloud Computing

- Internet connectivity is required to access cloud applications.
- Sensitive university data requires strong security controls.
- Cloud services may experience provider outages.
- Continuous usage may increase operational expenses.
- Dependence on a cloud provider may create vendor lock-in.

Conclusion:

Cloud computing is an effective solution for the university's existing infrastructure problems. A hybrid cloud deployment can reduce hardware expenses, improve resource utilization and

provide flexible computing capacity. It also supports centralized access to university services across all four campuses.

TASK 2 – Trace the Evolution and Hardware Evolution (3 Marks)

Trace how computing infrastructure evolved to make cloud computing possible.

- A timeline from mainframes, minicomputers and PCs, to client–server, clusters, grid computing, utility computing and cloud
- Hardware evolution: multi-core processors, commodity servers, high-capacity storage, high-speed networking and data centres
- How each stage solved a limitation of the previous one
- Which stage the university's current system corresponds to, with reasons

Evidence Expected

A labelled timeline or diagram, with a justified explanation of how hardware advances enabled the cloud.

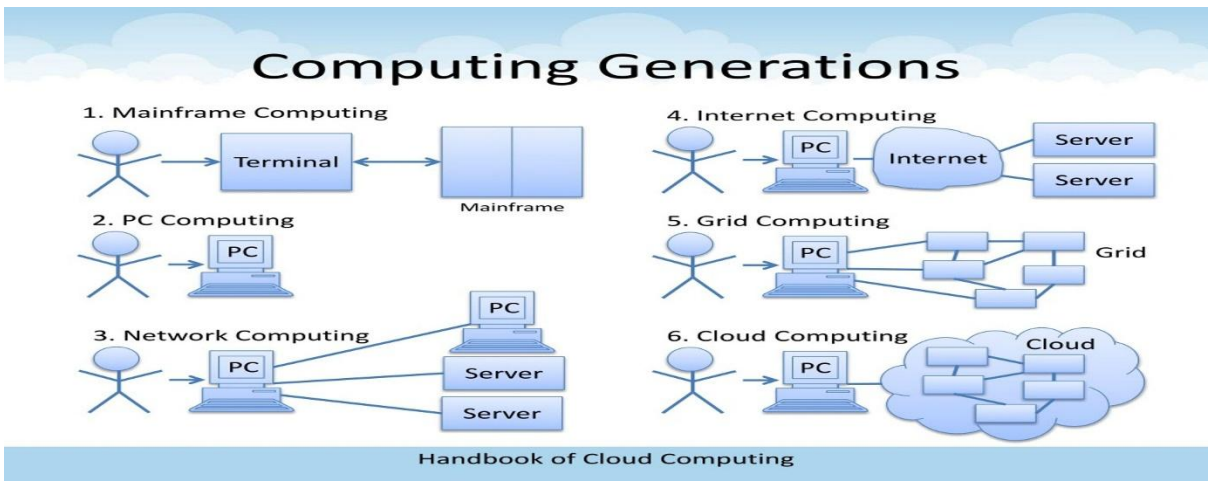
Answer:

TASK 2 – Trace the Evolution and Hardware Evolution (3 Marks)

2.1 Evolution of Computing Infrastructure

Computing infrastructure has evolved from large centralized computers to flexible cloud-based systems. Each stage improved processing power, accessibility, resource utilization and scalability.

2.2 Timeline of Computing Evolution



Stage	Period (Approx.)	Description	Limitation Solved
Mainframe Computing	1950s–1970s	Large centralized computers supported multiple users.	Replaced individual manual processing.
Minicomputers	1960s–1980s	Smaller and less expensive computers used by departments.	Reduced dependence on large mainframes.
Personal Computers	1970s–1990s	Individual users could perform computing tasks.	Improved personal accessibility.
Client–Server Computing	1980s–1990s	Clients requested services from centralized servers.	Enabled centralized data and application management.
Cluster Computing	1990s	Multiple connected computers worked together.	Improved processing performance and reliability.
Grid Computing	1990s–2000s	Distributed computers shared resources to solve large problems.	Enabled resource sharing across different locations.
Utility Computing	2000s	Computing resources were provided based on usage.	Introduced pay-per-use computing.
Cloud Computing	2006 onwards	Computing resources are delivered through the internet.	Provides elasticity, scalability and on-demand access.

2.3 Hardware Evolution

Hardware advancements played an important role in making cloud computing possible.

1. Multi-Core Processors

Multi-core processors contain multiple processing cores in a single CPU. They allow multiple operations to execute simultaneously.

University relevance: Multiple student requests can be processed efficiently during examination periods.

2. Commodity Servers

Commodity servers are standard, relatively low-cost servers that can be combined to create large computing environments.

University relevance: The university can use multiple standard servers instead of purchasing expensive specialized machines.

3. High-Capacity Storage

Modern storage technologies provide large storage capacity with faster data access.

University relevance: Student records, examination documents, learning materials and library files can be stored centrally.

4. High-Speed Networking

Modern networks support fast communication between servers, storage systems and users.

University relevance: Students from four campuses can access the same university services efficiently.

5. Data Centres

Data centres contain servers, storage, networking, cooling and power management systems.

University relevance: Cloud providers operate large data centres that offer computing resources without requiring the university to maintain its own physical infrastructure.

2.4 How Each Stage Improved Computing

Evolution	Improvement
Mainframe to Minicomputer	Reduced size and cost.
Minicomputer to PC	Provided individual computing access.
PC to Client–Server	Enabled centralized applications and databases.
Client–Server to Clusters	Improved processing and availability.
Clusters to Grid	Enabled distributed resource sharing.
Grid to Utility	Introduced usage-based computing.
Utility to Cloud	Added self-service, elasticity and internet-based access.

2.5 University's Existing Computing Stage

The university's existing infrastructure mainly follows the traditional client–server computing model.

Reasons:

- Each application runs on its own dedicated physical server.
- Applications depend on centralized servers for data processing.
- Resources are not shared efficiently.
- Average CPU utilization is only 15%.
- The infrastructure cannot efficiently handle peak workloads.

The university should move towards a virtualized cloud infrastructure to improve utilization and scalability.

Conclusion:

The development of processors, storage, networking and data centres made cloud computing possible. By adopting modern cloud infrastructure, the university can overcome the limitations of its traditional client–server environment and provide reliable services across all campuses.

TASK 3 – Analyse Internet Software Evolution (3 Marks)

Explain how internet software evolved and why it supports cloud services.

- Evolution from static web pages to dynamic web, Web 2.0, SOA, mashups and cloud-hosted applications
- Role of browsers, HTTP, AJAX and APIs in delivering software over the internet
- Comparison between desktop software, client–server software and web-delivered software
- How this evolution affects the design of the university's applications

Evidence Expected

A comparative explanation of software evolution stages, linked to the university's application design.

Answer:

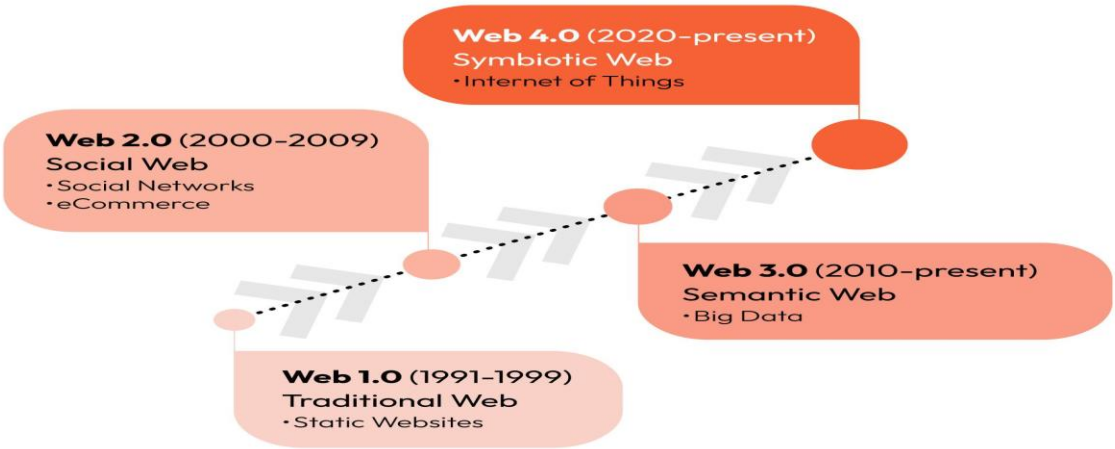
TASK 3 – Analyse Internet Software Evolution (3 Marks)

3.1 Introduction

Internet software evolution refers to the development of web technologies from simple static web pages to interactive applications and cloud-hosted services.

This evolution allows users to access software through browsers and mobile applications without installing complete software packages on individual computers.

3.2 Evolution of Internet Software



1. Static Web Pages (Web 1.0)

- Websites displayed fixed information.
- Users could only read the available content.
- Pages were mainly developed using HTML.

Example: A university website displaying department information and contact details.

2. Dynamic Web Applications

- Websites started displaying changing information.
- Server-side programming and databases were introduced.
- Users could submit forms and retrieve information.

Example: A student login portal displaying personal details.

3. Web 2.0

- Introduced interactive and user-generated content.
- Users could communicate and collaborate online.
- Technologies such as AJAX improved page interaction.

Example: An LMS where students can upload assignments and participate in discussions.

4. Service-Oriented Architecture (SOA)

SOA is a software architecture in which applications are developed as independent services that communicate through standard interfaces.

Example: The university can provide separate student information, library and fee-payment services.

5. Mashups

Mashups combine information or functionality from multiple services into one application.

Example: A university dashboard combining attendance information, examination schedules and library notifications.

6. Cloud-Hosted Applications

Applications are hosted on cloud infrastructure and accessed through the internet.

Example: A cloud-hosted student management system accessible from all four campuses.

3.3 Role of Web Technologies

Technology	**Function**	**University Usage**
Browser	Provides an interface to access web applications.	Students access the student portal.
HTTP/HTTPS	Transfers requests and responses between clients and servers.	Login and result retrieval.
AJAX	Updates parts of a webpage without reloading the entire page.	Displaying attendance updates.
APIs	Allow different applications to communicate.	Connecting mobile apps with student records.

JSON	Transfers structured data between applications.	Sending student details to mobile applications.

3.4 Comparison of Software Types

Feature	Desktop Software	Client–Server Software	Web-Delivered Software
Installation	Required on individual computers.	Client and server components required.	Usually accessed through a browser.
Accessibility	Limited to installed devices.	Available through connected clients.	Accessible through the internet.
Maintenance	Updated individually.	Server and client maintenance.	Centralized application updates.
Scalability	Limited.	Depends on server capacity.	Can use cloud auto-scaling.
Example	Desktop library software.	Traditional university management system.	Cloud-based student portal.

3.5 Effect on University Application Design

Internet software evolution changes how the university's applications should be designed.

1. The Student Information System should provide APIs for mobile and web access.
2. The LMS should support interactive learning and assignment submission.
3. The Library System should provide online book searching and availability.
4. The Exam Portal should support secure online examinations.
5. The Email System should support web-based access.

The applications should be divided into independent services instead of depending entirely on one large monolithic application.

Conclusion:

Internet software evolution has transformed simple websites into interactive and cloud-based applications. Technologies such as HTTP, AJAX, JSON and APIs enable the university to integrate its applications and provide centralized services to students and faculty.

TASK 4 – Apply Server Virtualization (5 Marks)

Apply server virtualization to consolidate the university's 12 physical servers.

- Explain how a hypervisor works, and distinguish Type-1 and Type-2 hypervisors, with examples
- Compare full virtualization, para-virtualization, OS-level virtualization (containers) and hardware-assisted virtualization
- Propose which applications should run as virtual machines and which as containers, with reasons

- Calculate the consolidation: with an average utilisation of 15% per server and a target host utilisation of 70%, estimate how many physical hosts are needed (show your assumptions and working)
- Discuss live migration, snapshots and resource over-commitment as benefits, and the risks of virtualization (noisy neighbour, hypervisor failure, security isolation)

Show your calculation. A final number without working will not receive full credit.

Evidence Expected

A justified virtualization plan with a worked consolidation calculation and discussion of benefits and risks.

Answer:

TASK 4 – Apply Server Virtualization (5 Marks)

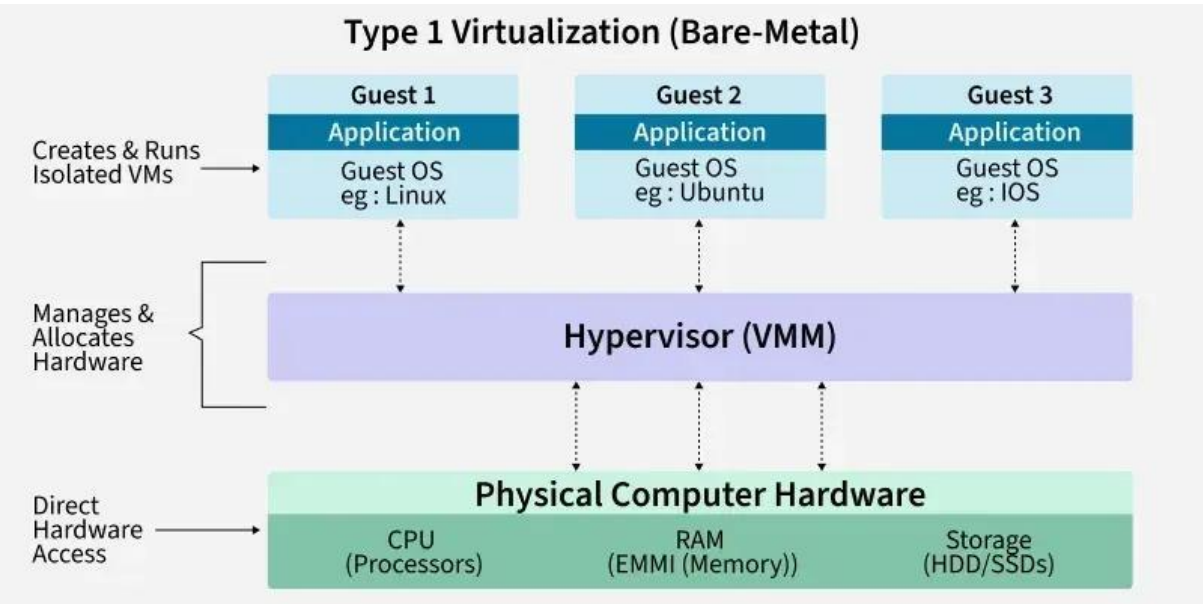
4.1 Introduction to Server Virtualization

Server virtualization is a technology that allows multiple virtual machines to run on a single physical server. Each virtual machine has its own operating system, CPU allocation, memory and storage.

A software layer called a hypervisor manages the virtual machines and distributes the physical hardware resources among them.

In the university scenario, 12 physical servers have an average CPU utilization of only 15%. Virtualization can consolidate these workloads onto fewer physical hosts.

4.2 Working of a Hypervisor



A hypervisor works as follows:

1. It runs on or above the physical hardware.
2. It creates multiple virtual machines.
3. It allocates CPU, RAM, storage and network resources.
4. It isolates virtual machines from each other.
5. It monitors and manages the running virtual machines.

4.3 Types of Hypervisors

Feature	Type-1 Hypervisor	Type-2 Hypervisor
Meaning	Runs directly on physical hardware.	Runs on top of a host operating system.
Performance	Generally higher.	Has additional host OS overhead.
Security	Smaller host software layer.	Depends on host OS security.
Usage	Enterprise data centres.	Testing and development.
Examples	VMware ESXi, Microsoft Hyper-V, Xen.	Oracle VirtualBox, VMware Workstation.

Recommended: Type-1 hypervisors are suitable for the university because they provide efficient resource management and are designed for enterprise environments.

4.4 Types of Virtualization

1. Full Virtualization

The hypervisor provides a complete virtual hardware environment for each virtual machine.

Example: Running Windows Server inside VMware ESXi.

2. Para-Virtualization

The guest operating system is aware of the hypervisor and communicates with it through special interfaces.

It can reduce virtualization overhead but requires compatible guest operating system support.

3. OS-Level Virtualization (Containers)

Multiple isolated containers share the host operating system kernel.

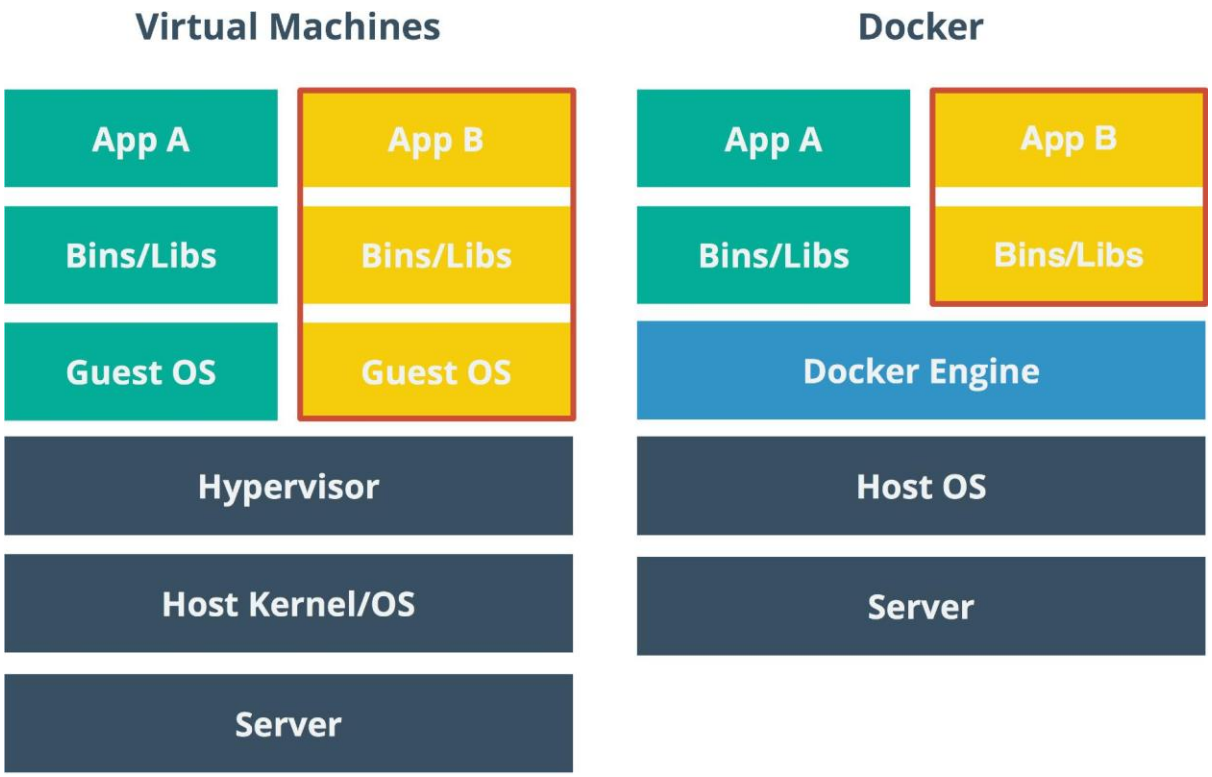
Example: Docker containers running university web applications.

4. Hardware-Assisted Virtualization

Modern processors provide virtualization support through technologies such as Intel VT-x and AMD-V.

These features improve virtual machine execution and reduce virtualization overhead.

4.5 Comparison Between Virtual Machines and Containers



Feature	Virtual Machines	Containers
Operating System	Each VM has its own guest OS.	Share the host OS kernel.
Resource Usage	Generally higher.	Generally lower.
Startup Time	Usually slower.	Usually faster.
Isolation	Strong isolation boundary when properly configured.	Process-level isolation.
Portability	Can be moved between compatible hypervisors.	Can be moved between compatible container environments.
Usage	Legacy applications and databases.	Modern web applications and APIs.
Example	VMware ESXi.	Docker.

4.6 Proposed Virtualization Plan for the University

The university currently has 12 physical servers.

The following plan can be adopted:

University Application	Proposed Environment	Reason
Student Information System	Virtual Machine	Supports legacy application requirements.
Learning Management System	Containers	Supports flexible scaling.

Library System	Virtual Machine	Maintains compatibility with existing software.
Exam Portal	Containers	Handles changing examination workloads.
Email System	SaaS	Reduces email server maintenance.
Database Server	Virtual Machine	Provides a controlled environment for legacy databases.

The university should use a combination of virtual machines and containers rather than migrating every application into the same environment.

4.7 Server Consolidation Calculation

Important – Show the full working for marks

Given:

- Total physical servers = 12
- Average CPU utilization per server = 15%
- Target host utilization = 70%

Step 1: Calculate the total CPU utilization.

$$\text{Total CPU Load} = 12 \times 15\%$$
$$= 180\%$$

This represents the equivalent CPU capacity of 1.8 fully utilized servers.

Step 2: Calculate the number of required physical hosts.

$$\text{Required Hosts} = \frac{\text{Total CPU Load}}{\text{Target Utilization}}$$
$$= \frac{180}{70}$$
$$= 2.57$$

Step 3: Round up to the next whole number.

$$\boxed{\text{Required Hosts} = 3}$$

Final calculation result

12

Existing servers

3

Calculated hosts

Theoretical consolidation: 12 servers to 3 hosts, a 75% reduction in physical server count.

Practical consideration: Three hosts are the mathematical minimum under the stated CPU assumptions, not necessarily a safe production design. RAM, storage, network capacity, peak loads, redundancy and maintenance must also be considered. For high availability, the university should evaluate additional host capacity, such as a four-host cluster.

4.8 Benefits of Virtualization

1. Live Migration

Running virtual machines can be moved from one physical host to another with minimal service interruption.

2. Snapshots

Snapshots capture the state of a virtual machine at a particular point in time. They are useful before updates and configuration changes.

3. Resource Over-Commitment

The hypervisor can allocate more virtual CPU or memory capacity than the physical resources available, based on expected usage.

It improves utilization but must be monitored carefully to avoid resource shortages.

4. Cost Reduction

Multiple virtual machines can run on fewer physical servers, reducing hardware, electricity and maintenance expenses.

4.9 Risks and Limitations of Virtualization

Risk	**Explanation**	**Mitigation**
Noisy Neighbour	One VM consumes excessive resources and affects other VMs.	Resource limits and monitoring.
Hypervisor Failure	Failure can affect multiple virtual machines.	High-availability cluster and backups.
Security Isolation	A vulnerability may allow attacks across virtual environments.	Regular patching and strong isolation.
Resource Overload	Excessive VM allocation reduces performance.	Capacity planning and monitoring.
Data Loss	Storage or snapshot failure may cause data loss.	Independent backups and disaster recovery.

Conclusion:

Server virtualization is an effective solution for consolidating the university's 12 physical servers. Based on the given average utilization and 70% target utilization, three physical hosts

are theoretically required. A production deployment should additionally provide sufficient redundancy, memory, storage and network capacity. Combining Type-1 hypervisors, virtual machines and containers will improve resource utilization, reduce infrastructure costs and support future growth.

TASK 5 – Apply Web Services (4 Marks)

Analyse web services to expose the university's functions to other systems.

- Overview of web services and their role in cloud and SOA
- Compare SOAP, WSDL, UDDI and RESTful services (HTTP methods, JSON/XML)
- Identify at least three university functions to expose as web services (for example, result lookup, fee payment, library search) and define the endpoint, method and sample request/response for each
- Justify whether SOAP or REST is more suitable for each function

Evidence Expected

A defined set of web services with sample request/response formats and a justified choice of SOAP or REST

Answer :

TASK 5 – Apply Web Services (4 Marks)

5.1 Introduction to Web Services

A web service is a software component that allows two different applications to communicate and exchange information over a network using standard protocols.

Web services are important in cloud computing because they allow applications to share data without depending on the same programming language or operating system.

For the university, web services can connect the Student Information System, Learning Management System, Library System, Exam Portal and mobile applications.

5.2 Role of Web Services in SOA

Service-Oriented Architecture (SOA) is an architectural approach in which software functionality is divided into independent services.

Each service performs a specific task and communicates with other services through standard interfaces.

Example:

- Student Service manages student information.

- Fee Service manages fee payments.
- Library Service manages book information.
- Result Service provides examination results.

These services can communicate without requiring all applications to be developed as one large program.

5.3 SOAP, WSDL, UDDI and REST

Technology	Description	University Usage
SOAP	Protocol for exchanging structured XML messages.	Secure and structured fee-related integrations.
WSDL	Describes the operations and message formats of a SOAP service.	Documents university service interfaces.
UDDI	Registry standard used to publish and discover web services.	Helps discover available services in a service registry.
REST	Architectural style that uses HTTP methods to access resources.	Mobile app and student portal APIs.

SOAP vs REST

Feature	SOAP	REST
Type	Protocol	Architectural style
Data Format	XML	JSON, XML and others
Communication	SOAP messaging	HTTP-based communication
Performance	Generally more message overhead.	Usually lightweight.
Complexity	More structured and complex.	Simpler to develop.
Usage	Enterprise integrations.	Web and mobile applications.

Recommended: REST is suitable for most university mobile and web APIs because it is lightweight and easy to integrate. SOAP can be retained where an existing enterprise integration requires its structured messaging features.

5.4 Three University Web Services

1. Student Result Lookup Service

This service allows students to view examination results using their register number and semester.

- Endpoint: /api/students/results
- HTTP Method: GET
- Purpose: Retrieve student examination results.
- Architecture: REST.

Request:

GET /api/students/results?regNo=23CSA001&semester=5

Response:


```
{
 "regNo": "23CSA001",
 "semester": 5,
 "studentName": "Arun",
 "result": "PASS",
 "cgpa": 8.5
}
```


2. Fee Payment Service

This service allows students to check outstanding fees and initiate payments.

- Endpoint: /api/fees/payment
- HTTP Method: POST
- Purpose: Process fee payment requests.
- Architecture: REST.

Request:


```
{
 "studentId": "23CSA001",
 "amount": 25000,
 "paymentMode": "UPI"
}
```


Response:


```
{
 "studentId": "23CSA001",
 "amount": 25000,
 "status": "SUCCESS",
 "transactionId": "TXN10025"
}
```


Note: This is an illustrative response. A real payment service must verify the payment gateway transaction before confirming success.

3. Library Search Service

This service allows students to search for books and check their availability.

- Endpoint: /api/library/books
- HTTP Method: GET
- Purpose: Retrieve book information.
- Architecture: REST.

Request:

GET /api/library/books?title=Cloud%20Computing

Response:


```
{
  "bookId": "B101",
  "title": "Cloud Computing",
  "author": "Rajkumar",
  "availability": "Available",
  "quantity": 5
}
```


5.5 HTTP Methods Used in REST

Method	Purpose	University Example
GET	Retrieve information.	View student results.
POST	Create or submit information.	Initiate a fee payment.
PUT	Update an existing resource.	Update student profile.
PATCH	Partially update a resource.	Change contact details.
DELETE	Remove a resource.	Delete an authorized record.

5.6 Advantages of Web Services

1. Supports communication between different applications.
2. Reduces duplication of data and functionality.
3. Supports mobile and web application integration.
4. Makes applications easier to maintain.
5. Allows affiliated colleges to access authorized university services.
6. Supports independent development and scaling of services.

Conclusion:

Web services provide an effective method for connecting the university's different applications. REST APIs are suitable for result lookup, fee payments and library searches because they provide lightweight communication. SOAP can be used when structured enterprise messaging is required. These services will also support future mobile application development and integration with affiliated colleges.

TASK 6 – Analyse the Service Models: IaaS, PaaS and SaaS (5 Marks) Analyse which service model fits each university application.

- Define IaaS, PaaS and SaaS, and show who manages which layer (hardware to application) in each model
- Categorise the five university applications (SIS, LMS, Library, Exam Portal, Email) into IaaS, PaaS or SaaS, with reasons
- Compare the models on control, cost, scalability, maintenance effort and vendor lockin
- Give one real provider example for each model

Present your **categorisation in a table**, but explain the **reasoning behind** each **decision**. A table alone is not sufficient.

Evidence Expected

A reasoned mapping of applications to service models, supported by a layer-wise responsibility comparison.

Answer:

TASK 6 – Analyse IaaS, PaaS and SaaS (5 Marks)

6.1 Introduction

Cloud computing provides different service models that determine how computing resources are managed by the cloud provider and the customer.

The three major service models are:

1. Infrastructure as a Service (IaaS).
2. Platform as a Service (PaaS).
3. Software as a Service (SaaS).

6.2 Infrastructure as a Service (IaaS)

IaaS provides virtualized computing infrastructure through the internet, including virtual machines, storage and networking.

The university can install its own operating systems and applications on the provided infrastructure.

Example: Amazon EC2, Microsoft Azure Virtual Machines.

University application: The legacy Student Information System can run on cloud virtual machines because it may require specific operating system and software configurations.

6.3 Platform as a Service (PaaS)

PaaS provides a complete development and deployment platform.

The cloud provider manages the underlying infrastructure, operating system and runtime environment, while developers manage their application code and data.

Example: Google App Engine, Azure App Service.

University application: The university can develop and deploy its student mobile application APIs without managing physical servers.

6.4 Software as a Service (SaaS)

SaaS provides ready-to-use software through the internet.

The provider manages the complete application infrastructure, maintenance and updates.

Example: Google Workspace, Microsoft 365.

University application: The university can use a cloud email platform instead of maintaining its own email servers.

6.5 Responsibility Layers

Component	IaaS	PaaS	SaaS
Physical Hardware	Provider	Provider	Provider
Networking	Provider	Provider	Provider
Storage Infrastructure	Provider	Provider	Provider
Virtualization	Provider	Provider	Provider
Operating System	Customer	Provider	Provider
Middleware / Runtime	Customer	Provider	Provider
Application	Customer	Customer	Provider
Application Data	Customer	Customer	Customer-controlled data and settings

The table shows the typical responsibility split. Exact responsibilities vary by provider and service.

6.6 Categorization of Five University Applications

Application	Selected Model	Reason
Student Information System (SIS)	IaaS	Legacy software may require control over the OS and application environment.
Learning Management System (LMS)	PaaS	Supports application customization and flexible deployment.
Library System	IaaS	Existing library software and databases may need specific configurations.
Exam Portal	PaaS	Allows quick deployment and scaling during examinations.
Email System	SaaS	Ready-made email services reduce administration and maintenance.

Explanation of Application Mapping

1. Student Information System – IaaS

The SIS contains important student details, attendance, academic records and personal information. By selecting IaaS, the university can maintain control over the operating system, database and application configuration while avoiding the purchase of physical servers.

2. Learning Management System – PaaS

The LMS requires regular application updates, assignment management and student interactions. PaaS allows developers to deploy updates without managing the underlying servers.

3. Library System – IaaS

The existing library system may depend on legacy software and database configurations. IaaS allows the university to migrate the application while maintaining compatibility with its existing environment.

4. Exam Portal – PaaS

The exam portal experiences sudden increases in usage during examination periods. PaaS supports application deployment and scaling without requiring the university to manage physical infrastructure.

5. Email System – SaaS

Email is a standard service that does not require the university to develop its own application. SaaS platforms provide email, calendars, security updates and storage.

6.7 Comparison of IaaS, PaaS and SaaS

Factor	**IaaS**	**PaaS**	**SaaS**
Control	High	Medium	Low
Initial Cost	Low hardware investment	Low infrastructure investment	Subscription-based
Scalability	High	High	Provider-managed
Maintenance	More customer effort	Shared responsibility	Mostly provider-managed
Flexibility	High	Medium to high	Limited customization
Vendor Lock-in	Depends on architecture	Can be higher due to platform-specific services	Can be higher due to proprietary data and workflows

6.8 Real Cloud Provider Examples

Service Model	**Provider**	**Service**
IaaS	AWS	Amazon EC2
PaaS	Microsoft Azure	Azure App Service
SaaS	Google	Google Workspace

Conclusion:

IaaS, PaaS and SaaS provide different levels of control and management. For the university, IaaS is appropriate for legacy applications, PaaS is suitable for customizable applications that need scalability, and SaaS is suitable for standard services such as email. A combination of these models provides flexibility and reduces infrastructure management.

TASK 7 – Examine XaaS (3 Marks)

Examine the broader XaaS (Anything-as-a-Service) concept for the university.

- Explain XaaS and describe at least four variants (for example DaaS, BaaS/MBaaS, FaaS, DBaaS, Security-as-a-Service, Disaster-Recovery-as-a-Service)
- Select two XaaS offerings that would benefit the university, and justify them
- Discuss the risks of over-reliance on multiple 'as-a-Service' providers

Evidence Expected

An analysis of XaaS variants, with justified selection of two offerings for the university.

Answer:

TASK 7 – Examine XaaS (3 Marks)

7.1 Introduction to XaaS

XaaS stands for Anything as a Service. It is a cloud computing concept in which different IT resources, applications and services are delivered through the internet on a subscription or pay-per-use basis.

Instead of purchasing and maintaining every IT component, organizations can obtain services from specialized cloud providers.

7.2 Different Types of XaaS

1. Database as a Service (DBaaS)

DBaaS provides database services through the cloud. The provider manages database infrastructure, maintenance and backups.

Example: Amazon RDS.

University usage: Store student records, examination details and fee information.

2. Disaster Recovery as a Service (DRaaS)

DRaaS provides cloud-based disaster recovery solutions that help organizations recover applications and data after unexpected failures.

Example: AWS Elastic Disaster Recovery.

University usage: Recover university applications if the primary data centre becomes unavailable.

3. Security as a Service (SECaaS)

SECaaS provides security solutions through cloud platforms.

Example: Cloudflare security services.

University usage: Protect student portals against malicious traffic and unauthorized access.

4. Function as a Service (FaaS)

FaaS allows developers to execute small pieces of code without managing servers.

Example: AWS Lambda.

University usage: Automatically send notifications when examination results are published.

5. Backend as a Service (BaaS / MBaaS)

BaaS provides ready-made backend services such as authentication, databases and notifications for mobile and web applications.

Example: Firebase.

University usage: Support student mobile application login and notifications.

6. Desktop as a Service (DaaS)

DaaS provides virtual desktop environments through the internet.

Example: Amazon WorkSpaces.

University usage: Provide faculty and administrative staff with remote access to university desktops.

7.3 Recommended XaaS Offerings

1. Database as a Service (DBaaS)

DBaaS is recommended because the university maintains information about approximately 25,000 students and 2,000 staff.

Benefits:

- Reduces database infrastructure maintenance.
- Provides automated backups.
- Supports database monitoring.
- Allows storage and computing capacity to grow.
- Improves database management.

2. Disaster Recovery as a Service (DRaaS)

DRaaS is recommended because the existing university infrastructure does not have a disaster recovery site.

Benefits:

- Provides recovery of important university applications.
- Protects academic and financial records.
- Reduces downtime during unexpected failures.
- Supports backup and recovery in another geographical region.

7.4 Risks of Depending on Multiple XaaS Providers

Risk	**Explanation**	**Solution**
Vendor Lock-in	Difficult to move services between providers.	Use open standards and portable data formats.
Security Issues	Data may be stored across different platforms.	Apply encryption and access controls.
Service Downtime	Provider failures may interrupt university services.	Maintain backup and recovery plans.
Integration Complexity	Different providers may use different interfaces.	Use standard APIs.

Cost Increase	Multiple subscriptions may increase expenses.	Monitor usage and review contracts.

Conclusion:

XaaS provides flexible and cost-effective IT services without requiring organizations to manage every component. DBaaS and DRaaS are the most beneficial options for the university because they improve database management and provide protection against unexpected failures.

TASK 8 – Design the Cloud Solution Architecture (5 Marks)

Bloom's Level: Level 6 – Create | *Action verbs: Design, Construct, Propose*

Create a complete cloud architecture for the university by combining virtualization, web services and service models.

- An architecture diagram showing users, load balancer, web/application tier, database tier, storage, API layer and DR region
- The deployment model and the service model chosen for each component
- How elasticity (auto-scaling) will handle admissions and exam peaks
- Basic security and availability measures (access control, encryption, backup, multizone deployment)

Evidence Expected

A labelled architecture diagram with written justification for each component and design decision.

Answer:

TASK 8 – Design the Cloud Solution Architecture (5 Marks)

8.1 Introduction

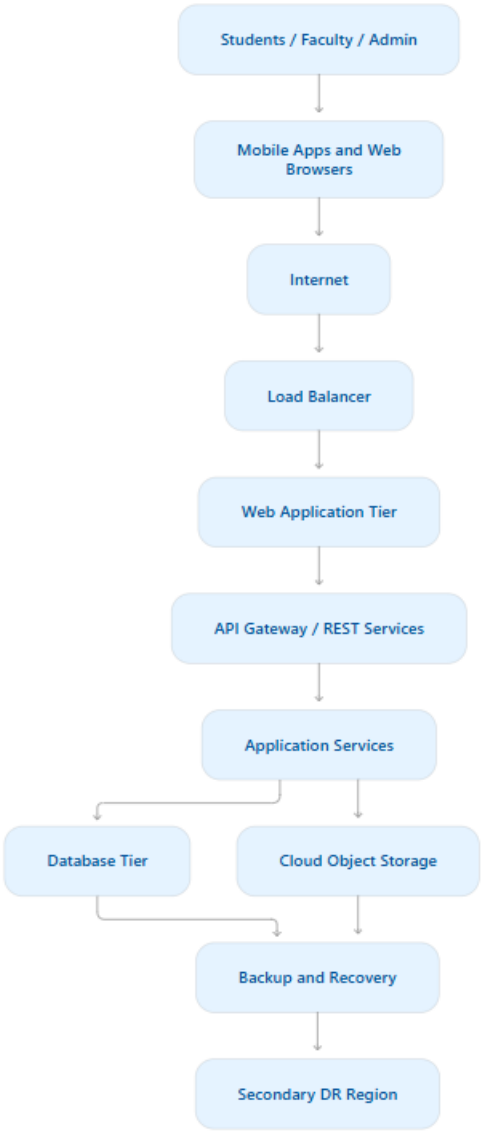
Cloud architecture is the arrangement of cloud-based components such as servers, databases, storage, networking and applications to deliver services efficiently.

The proposed architecture for the university should support 25,000 students, 2,000 staff members and four campuses.

The design should provide scalability, security, availability and disaster recovery.

8.2 Proposed Cloud Architecture Diagram

Figure 8.1 – Proposed Hybrid Cloud Architecture for University Management System



8.3

Explanation of Architecture Components

1. Users

Students, faculty members and administrators access university services through browsers and mobile applications.

2. Internet

The internet connects users from all four campuses to the cloud infrastructure.

3. Load Balancer

The load balancer distributes incoming requests across multiple application servers.

It prevents a single server from receiving excessive traffic.

4. Web Application Tier

This layer hosts the university's web applications, including the student portal, LMS and examination interface.

5. API Gateway

The API gateway manages requests between mobile applications, external colleges and university services.

It supports authentication, request routing, rate limiting and API monitoring.

6. Application Services

This layer contains the individual university applications:

- Student Information Service.
- Learning Management Service.
- Library Service.
- Examination Service.
- Fee Management Service.

7. Database Tier

The database stores student details, examination results, attendance, library information and fee records.

8. Cloud Storage

Object storage is used for documents, assignment files, examination materials and backups.

9. Backup and Recovery

Important application data and databases are backed up regularly.

10. Disaster Recovery Region

A secondary geographical cloud region stores recovery copies of critical information and supports service restoration if the primary region fails.

8.4 Deployment and Service Models

Architecture Component	**Deployment Model**	**Service Model**
Student Portal	Public Cloud	PaaS
Legacy SIS	Private Cloud / Hybrid	IaaS
LMS	Public Cloud	PaaS
Library Application	Hybrid Cloud	IaaS
Email	Public Cloud	SaaS
Database	Hybrid Cloud	DBaaS
File Storage	Public Cloud	Storage as a Service
Backup and Recovery	Hybrid Cloud	DRaaS

The hybrid deployment model is selected to balance sensitive data protection with the scalability of public cloud infrastructure.

8.5 Elasticity and Auto-Scaling

Auto-scaling automatically increases or decreases computing resources according to workload.

For example:

- During normal days, a smaller number of application instances can handle requests.
- During admissions, additional instances are launched.
- During examination result publication, more computing resources are allocated.
- After peak usage decreases, unnecessary instances are removed.

This helps the university maintain performance without continuously paying for maximum capacity.

8.6 Security Measures

Security Measure	Purpose
Identity and Access Management	Ensures only authorized users access university systems.
Multi-Factor Authentication	Provides additional login protection.
Data Encryption	Protects data during transfer and storage.
Firewall	Controls unauthorized network traffic.
Regular Backups	Protects against data loss.
Role-Based Access Control	Provides access based on student, faculty and administrator roles.
Security Monitoring	Detects suspicious activities.

8.7 Availability and Disaster Recovery

The university should implement:

1. Multi-zone application deployment.
2. Regular database backups.
3. Replication of critical data.
4. Health monitoring of application servers.
5. Automatic failover where supported.
6. A secondary disaster recovery region.
7. Periodic disaster recovery testing.

Conclusion:

The proposed hybrid cloud architecture combines virtualization, web services, databases and cloud service models. The load balancer and auto-scaling support high workloads, while encryption, access control and backup protect university information. A secondary disaster recovery region provides additional protection against major infrastructure failures.

TASK 9 – Hands-on Implementation and Documentation (4 Marks)

Implement a small proof-of-concept of your design using a free-tier cloud account or a local virtualization tool.

- Create a virtual machine or container (using AWS/Azure/GCP free tier, VirtualBox, or Docker) and host a simple web page or REST API for one university function
- Deploy a SaaS or PaaS component (for example a cloud-hosted database, a serverless function or a static site)
- Test the web service with a client (browser, Postman or curl) and record the output
- Provide screenshots, configuration steps and any issues faced

Evidence Expected

Annotated screenshots and clear, reproducible steps showing that the proof-of-concept works.

Answer:

TASK 9 – Hands-on Implementation and Documentation (4 Marks)

9.1 Aim

To implement a small cloud-based university application using Docker virtualization and deploy a simple REST API that provides student information.

9.2 Software and Tools Required

Tool	Purpose
Python	Backend programming
Flask	Creating REST APIs
Docker	Containerization
VS Code	Writing and editing code
Postman / Browser	Testing REST APIs
Cloud Platform	Hosting the application

9.3 Implementation

In this experiment, a simple Student Information REST API is created using Python Flask and deployed inside a Docker container.

The API provides student details through an HTTP GET request.

Step 1: Create a Project Folder

Create a folder named:

UniversityCloudAPI

Inside the folder, create these files:

UniversityCloudAPI/

```
├── app.py
├── requirements.txt
└── Dockerfile
```

Step 2: Create the Flask Application

Open app.py and enter the following code:

```
from flask import Flask, jsonify


app = Flask(__name__)


students = [
    {
        "id": 1,
        "name": "Arun",
        "department": "Computer Science",
        "semester": 5
    },
    {
        "id": 2,
        "name": "Priya",
        "department": "Information Technology",
        "semester": 6
    }
]
```

```
@app.route("/")
def home():
    return jsonify({
        "message": "University Cloud API is Running"
    })

@app.route("/api/students")
def get_students():
    return jsonify(students)

@app.route("/api/students/<int:student_id>")
def get_student(student_id):
    for student in students:
        if student["id"] == student_id:
            return jsonify(student)

    return jsonify({
        "error": "Student Not Found"
    }), 404

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 3: Create requirements.txt

```
Flask
```

This file specifies the Python library required to run the application.

Step 4: Create Dockerfile

FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]

The Dockerfile defines how the application container is created.

Step 5: Install and Build Docker Image

Open the terminal inside the project folder.

Build the Docker image:

docker build -t university-api .

Check available Docker images:

docker images

Step 6: Run the Docker Container

docker run -d -p 5000:5000 --name university-container university-api

Check the running container:

docker ps

The application is now accessible through:

http://localhost:5000

Step 7: Test the REST API

Open the browser or Postman.

Test Case 1: Check Application Status

Request:

GET http://localhost:5000/

Expected Response:


```
{
  "message": "University Cloud API is Running"
}
```


Test Case 2: Retrieve All Students

Request:

GET http://localhost:5000/api/students

Expected Response:


```
[
  {
    "id": 1,
    "name": "Arun",
    "department": "Computer Science",
    "semester": 5
  },
  {
    "id": 2,
    "name": "Priya",
    "department": "Information Technology",
    "semester": 6
  }
]
```


Test Case 3: Retrieve a Specific Student

Request:

GET http://localhost:5000/api/students/1

Expected Response:


```
{
  "id": 1,
  "name": "Arun",
  "department": "Computer Science",
  "semester": 5
}
```


Test Case 4: Test Invalid Student ID

Request:

GET http://localhost:5000/api/students/10

Expected Response:

```
{
  "error": "Student Not Found"
}
```

Expected HTTP status codes: 200 for successful requests and 404 for an unknown student.

9.4 Deploying a PaaS Component

To demonstrate PaaS, the Flask application can be deployed using a managed application platform such as Microsoft Azure App Service.

Procedure:

1. Create an Azure account.
2. Open Azure App Service.
3. Create a Web App with a supported Python runtime.
4. Configure the application deployment.
5. Upload or connect the Flask project.
6. Start the application.
7. Open the generated application URL.
8. Test the student API using a browser or Postman.

Availability and pricing of free-tier resources depend on the provider's current offer and region.

9.5 Screenshots to Include

The following screenshots should be captured during actual execution.

Figure 9.1 – Project Structure

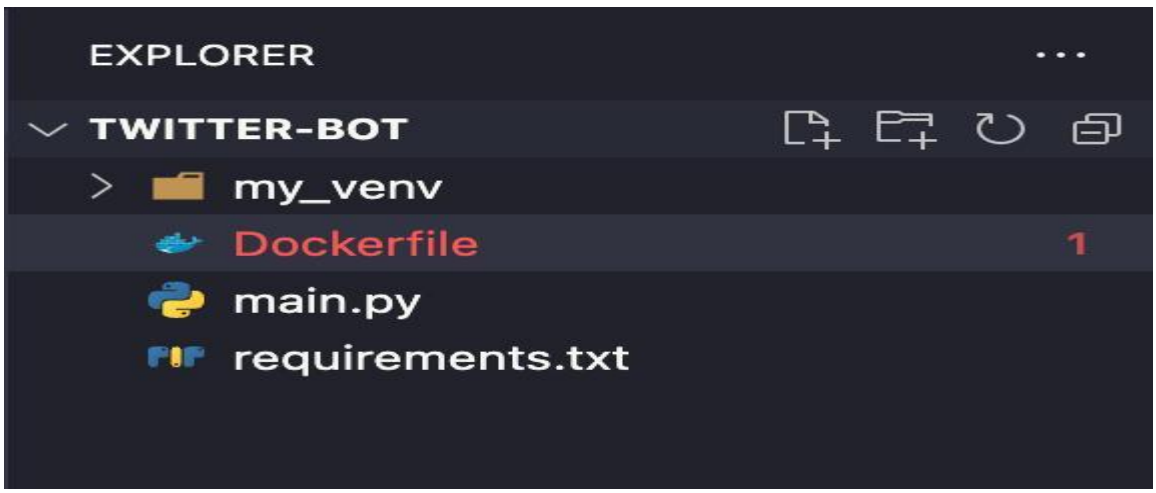


Figure 9.2 – Docker Image Build

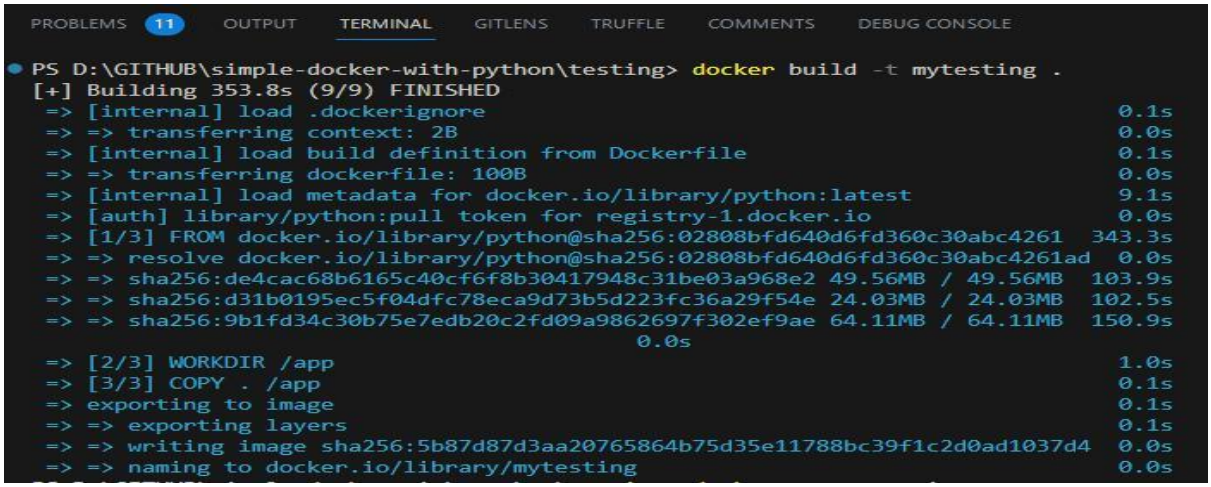


Figure 9.3 – Running Docker Container

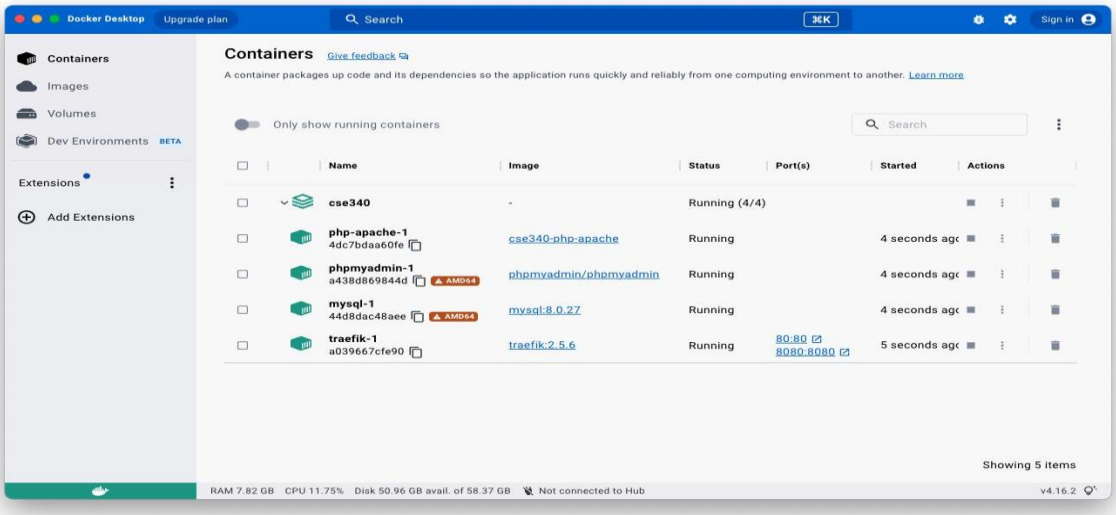


Figure 9.4 – REST API Output

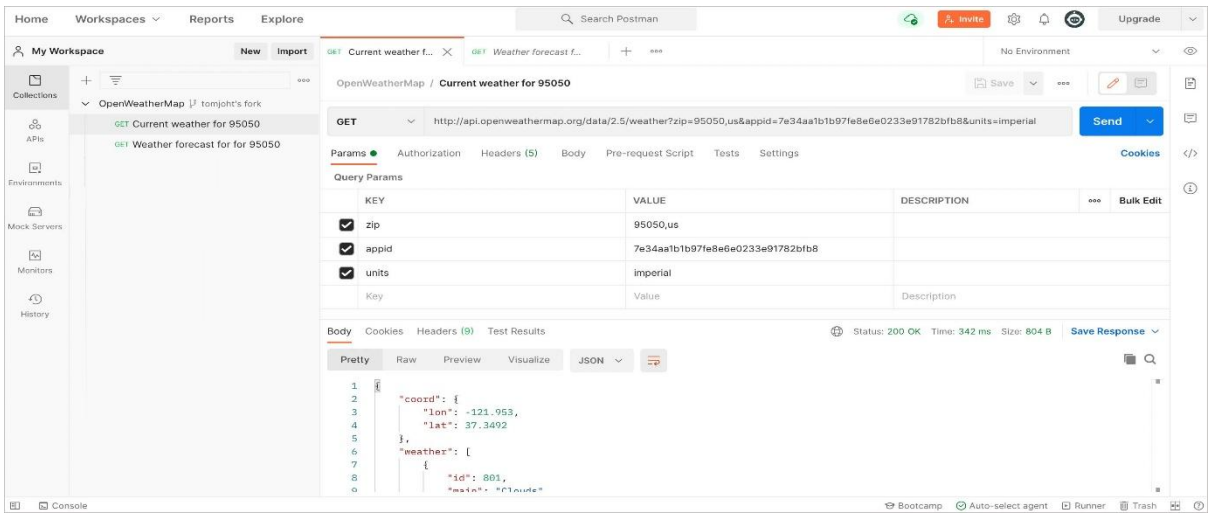
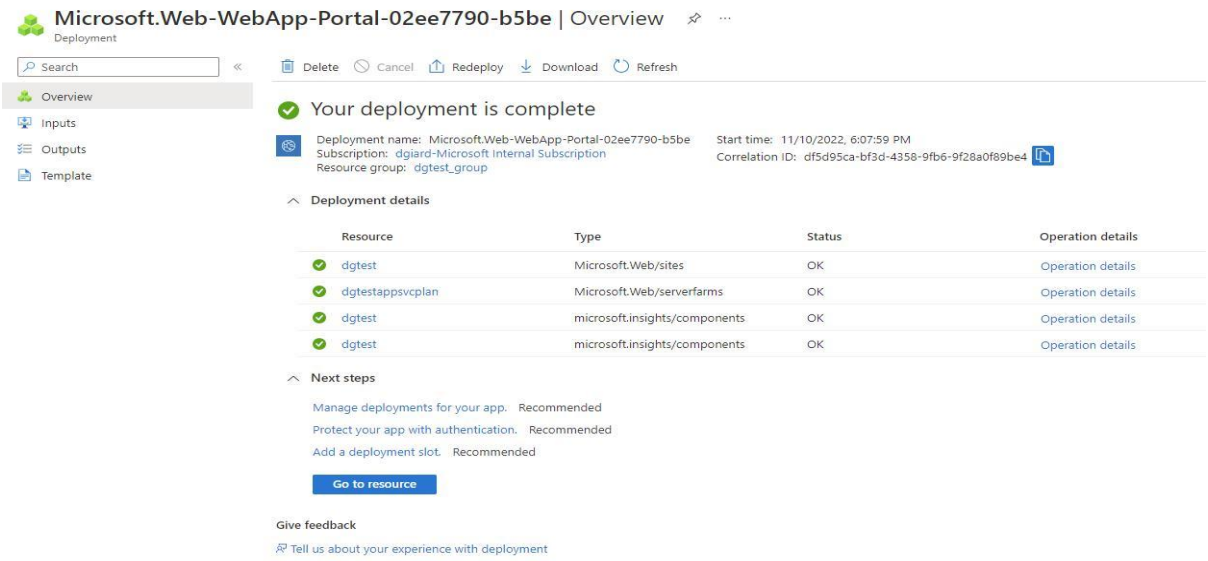


Figure 9.5 – Cloud Deployment



These are visual references for the screenshots to capture, not proof of your actual execution. Replace them with your own screenshots.

9.6 Issues Faced and Solutions

Issue	Solution
Flask module not found	Install Flask using pip.
Port already in use	Use another available port.
Docker build failure	Check Dockerfile and dependencies.
Container not starting	Check logs using docker logs university-container.
API not accessible	Verify port mapping and firewall settings.

9.7 Result

A simple university student information REST API was designed using Python Flask and packaged into a Docker container. The API supports student retrieval and error handling. It can also be deployed using a managed PaaS platform.

Conclusion:

This practical implementation demonstrates how virtualization and cloud technologies can be used to deploy university applications. Docker provides application isolation and portability, while REST APIs allow other university applications to retrieve student information.

TASK 10 – Evaluate and Recommend the Final Solution (7 Marks)

Evaluate your design and recommend the final cloud solution for the university.

- Compare on-premise, IaaS-based and PaaS/SaaS-based options on cost (CapEx vs OpEx), scalability, availability, security and effort

- Estimate a simple 3-year cost comparison using stated assumptions
- Identify the main risks (vendor lock-in, data privacy, downtime, skills gap) and give mitigation steps
- Provide your final recommendation and a short migration roadmap (which application moves first, and why)

Your recommendation must be supported by the analysis in the previous tasks. Choosing a popular cloud provider without justification is not sufficient.

Evidence Expected

A technically justified recommendation, supported by a cost comparison, risk analysis and migration roadmap.

Answer:

TASK 10 – Evaluate and Recommend the Final Solution (7 Marks)

10.1 Introduction

The university currently operates 12 physical servers with approximately 15% average CPU utilization. This creates unnecessary infrastructure expenses and makes it difficult to handle sudden increases in workload.

Three possible solutions are evaluated:

1. Continue with the existing on-premise infrastructure.
2. Migrate applications to an IaaS-based cloud environment.
3. Adopt a combination of PaaS and SaaS services.

The final solution should provide scalability, security, availability and cost efficiency.

10.2 Comparison of Deployment Options

Factor	On-Premise	IaaS	PaaS / SaaS
Cost	High initial investment.	Usage-based infrastructure cost.	Subscription or usage-based cost.
Scalability	Requires hardware procurement.	Virtual machines can be increased.	Managed scaling options.
Availability	Depends on university infrastructure.	Provider infrastructure and redundancy options.	Provider-managed infrastructure.
Security	Full infrastructure control.	Shared responsibility.	Provider manages more infrastructure layers.
Maintenance	University manages everything.	University manages OS and applications.	Less infrastructure maintenance.
Flexibility	High hardware control.	High infrastructure control.	Depends on platform.
Management Effort	High	Medium	Low

10.3 Three-Year Cost Comparison

The following is an illustrative planning estimate in Indian Rupees. Actual costs depend on hardware quotations, cloud region, storage, network traffic, licensing and service usage.

Assumptions

- University has 12 existing physical servers.
- Three-year comparison period.
- 25,000 students and 2,000 staff.
- Costs include infrastructure and basic administration.
- Existing data centre space is available.
- Figures are planning estimates, not provider quotations.

Option 1: On-Premise Infrastructure

Expense	Estimated Cost
Server hardware and upgrades	₹18,00,000
Storage and networking	₹6,00,000
Electricity and cooling (3 years)	₹7,20,000
Maintenance and support (3 years)	₹4,80,000
Backup infrastructure	₹3,00,000
Total	₹39,00,000

Option 2: IaaS-Based Cloud

Expense	Estimated Cost
Virtual machine usage (3 years)	₹15,00,000
Cloud storage (3 years)	₹4,50,000
Networking and data transfer	₹3,00,000
Backup and monitoring	₹3,00,000
Migration and administration	₹4,50,000
Total	₹30,00,000

Option 3: PaaS/SaaS-Based Cloud

Expense	Estimated Cost
Managed application platforms	₹9,00,000
Managed database services	₹4,50,000
Email and collaboration subscriptions	₹3,00,000
Storage and backup	₹3,00,000
Migration and administration	₹4,50,000
Total	₹24,00,000

Three-Year Cost Comparison

Estimated three-year infrastructure cost

Illustrative estimates in Indian Rupees.

₹0 lakh₹15 lakh₹30 lakh₹45 lakhOn-premiseIaaSPaaS/SaaS

Cost Savings Calculation

IaaS Savings:

$$ 39,00,000-30,00,000=9,00,000 $$
$$ \text{Savings Percentage}=\frac{9}{39}\times100 $$
$$ =23.08\% $$

PaaS/SaaS Savings:

$$ 39,00,000-24,00,000=15,00,000 $$
$$ \text{Savings Percentage}=\frac{15}{39}\times100 $$
$$ =38.46\% $$

Therefore, based on these assumptions, the PaaS/SaaS option offers the lowest estimated three-year cost.

10.4 Main Risks and Mitigation Strategies

Risk	Impact	Mitigation
Vendor Lock-in	Difficult to migrate to another cloud provider.	Use open APIs and portable data formats.
Data Privacy	Unauthorized access to student information.	Encryption and role-based access control.
Cloud Downtime	University applications may become unavailable.	Multi-zone deployment and disaster recovery.
Skills Gap	Staff may lack cloud administration knowledge.	Provide technical training.
Cost Overrun	Uncontrolled resource consumption increases bills.	Budget alerts and usage monitoring.
Data Loss	Important student records may be lost.	Automated backups and recovery testing.

10.5 Final Recommendation

The university should adopt a hybrid cloud solution using IaaS, PaaS and SaaS rather than completely depending on one service model.

The recommended approach is:

- Use IaaS for legacy Student Information and Library applications.
- Use PaaS for the LMS, Exam Portal and new university APIs.
- Use SaaS for email and collaboration.
- Use DBaaS for managed database operations.
- Use DRaaS for disaster recovery.

- Use virtualization to consolidate existing workloads.

This combination provides flexibility while allowing the university to retain control over applications that require special configurations.

10.6 Migration Roadmap

Phase	Activity	Expected Outcome
Phase 1	Analyse existing applications and databases.	Migration requirements identified.
Phase 2	Set up cloud infrastructure and security.	Cloud environment prepared.
Phase 3	Migrate the Library System and test it.	First application successfully migrated.
Phase 4	Migrate SIS and databases.	Student information available in cloud.
Phase 5	Deploy LMS and Exam Portal using PaaS.	Scalable applications available.
Phase 6	Move email to SaaS.	Reduced email server maintenance.
Phase 7	Configure backup and DR services.	Improved recovery capability.
Phase 8	Perform performance and security testing.	Migration validated.

Conclusion:

After evaluating the three deployment options, the hybrid cloud model with a combination of IaaS, PaaS and SaaS is recommended. It provides a balance between cost, flexibility, security and scalability. The estimated three-year cost of ₹24 lakh for the PaaS/SaaS option is lower than the illustrative on-premise and IaaS estimates. A phased migration will reduce operational risks and allow the university to move applications gradually.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The university will now launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10 times the current peak, availability must be 99.9%, and the operating budget is limited.

Re-evaluate your original design in response to these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

Impact Analysis – 2 Marks Discuss:

- What has changed from the original problem
- Which original assumptions (load, users, availability, cost) are affected
- Why the web services and APIs now become critical
- How multi-tenant access by 40 colleges affects security and isolation

Evidence Expected: A clear analysis of how the changed requirements affect the original cloud design.

Evaluation of the Original Design – 2 Marks Discuss:

- Whether the chosen service models and virtualization plan can handle a 10x load
- Whether auto-scaling and the single-region deployment are sufficient for 99.9% availability (about 8.76 hours of downtime allowed per year)
- Whether cost remains acceptable under the new load
- Whether another service model or XaaS offering would give better overall results

Evidence Expected: A technically supported evaluation of the original design under the new conditions.

Adaptation or Redesign – 2 Marks Discuss:

- Decide whether to retain, modify or redesign the solution
- Describe the revised architecture (for example API gateway, container orchestration, serverless components, multi-zone deployment or a hybrid model)
- Explain what changed in the service model mix and in the virtualization approach
- If you retain the original design, give sufficient technical justification for why no change is required

Evidence Expected: An appropriate design response to the changed requirement, with clear technical reasoning.

Evidence-Based Defence – 2 Marks Discuss:

- Capacity and availability calculations
- Cost comparison between the original and the revised design
- Security and multi-tenancy considerations
- Trade-offs such as vendor lock-in, complexity and migration effort

Evidence Expected: A well-supported final decision that follows logically from the analysis.

Answer:

TASK 11 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

11.1 Introduction

The university initially planned to migrate its existing management system to a hybrid cloud environment.

However, a new requirement changes the original design.

The university now plans to:

- Launch a mobile application.
- Provide student-record services to 40 affiliated colleges.
- Provide online fee-payment services.
- Handle 10 times the current peak workload.

- Maintain 99.9% service availability.
- Operate within a limited budget.

These requirements increase the need for scalability, security, API management and high availability.

11.2 Impact Analysis (2 Marks)

A. Changes in Requirements

Original Requirement	New Requirement	Impact
Four university campuses	Four campuses and 40 affiliated colleges	More users and organizations.
Web-based applications	Web and mobile applications	Additional API requirements.
Existing peak workload	10 times current peak	Higher computing capacity.
Traditional access	External college access	Stronger access management.
Existing availability approach	99.9% availability	Improved redundancy required.
Limited application integration	Open student-record and payment services	API gateway becomes essential.

B. Effect on Original Assumptions

1. Increased Workload

The original architecture was designed for the existing university workload. The new 10× requirement means more processing capacity is needed.

2. Increased Number of Users

Forty affiliated colleges will access university services. The architecture must support more users and organizations.

3. Mobile Application Integration

The mobile application needs APIs to retrieve student information, check fee status and process payment requests.

4. Availability Requirement

The university now requires 99.9% availability. A single-region design with no redundancy is insufficient.

5. Security and Tenant Isolation

Each affiliated college must access only its authorized student records. Data must be separated between colleges.

C. Importance of APIs

APIs become critical because they allow external colleges and mobile applications to communicate with university systems.

The API gateway should provide:

- Authentication.
- Authorization.
- Request routing.
- Rate limiting.
- API monitoring.
- Request validation.

D. Multi-Tenant Security

Multi-tenancy allows multiple colleges to use the same application infrastructure while keeping their information logically separated.

For example:

- College A can access only College A's student records.
- College B can access only College B's student records.
- University administrators can access authorized records across colleges.

This can be implemented using tenant IDs, role-based access control and database-level access restrictions.

11.3 Evaluation of the Original Design (2 Marks)

A. Can the Original Virtualization Plan Handle 10× Load?

The original design calculated three physical hosts based on an average CPU utilization of 15% and target utilization of 70%.

However, the new requirement increases workload by ten times.

Original CPU workload:

$$ 12\times15\%=180\% $$

New CPU workload:

$$ 180\%\times10=1800\% $$

Required hosts at 70% target utilization:

$$ \frac{1800}{70}=25.71 $$
$$ \boxed{26\text{ host equivalents}} $$

Therefore, the original three-host plan cannot handle the new workload by itself.

This is a theoretical CPU-based estimate. Actual capacity planning must also consider peak request rates, memory, storage, database throughput and network traffic.

B. Is Single-Region Deployment Sufficient?

A single-region architecture may become unavailable if the region experiences a major outage.

The new design should use:

- Multiple availability zones in the primary region.
- Database replication.
- Load balancing.
- Automated health checks.
- A secondary disaster recovery region.

C. Availability Calculation

Required availability = 99.9%.

$$\text{Downtime}=365\times24\times60$$
$$=525600\text{ minutes}$$

Permitted downtime:

$$525600\times\frac{0.1}{100}$$
$$=525.6\text{ minutes}$$
$$\boxed{8.76\text{ hours per year}}$$

Therefore, the system must be designed to keep annual downtime within approximately 8 hours and 46 minutes.

D. Cost Evaluation

The original infrastructure estimate may no longer be sufficient because the university now requires:

- More computing resources.
- Additional storage.
- API gateway services.
- Database scaling.
- Security services.
- Multi-zone availability.
- Disaster recovery.

Therefore, the original design must be modified to control costs while supporting increased demand.

11.4 Adaptation or Redesign (2 Marks)

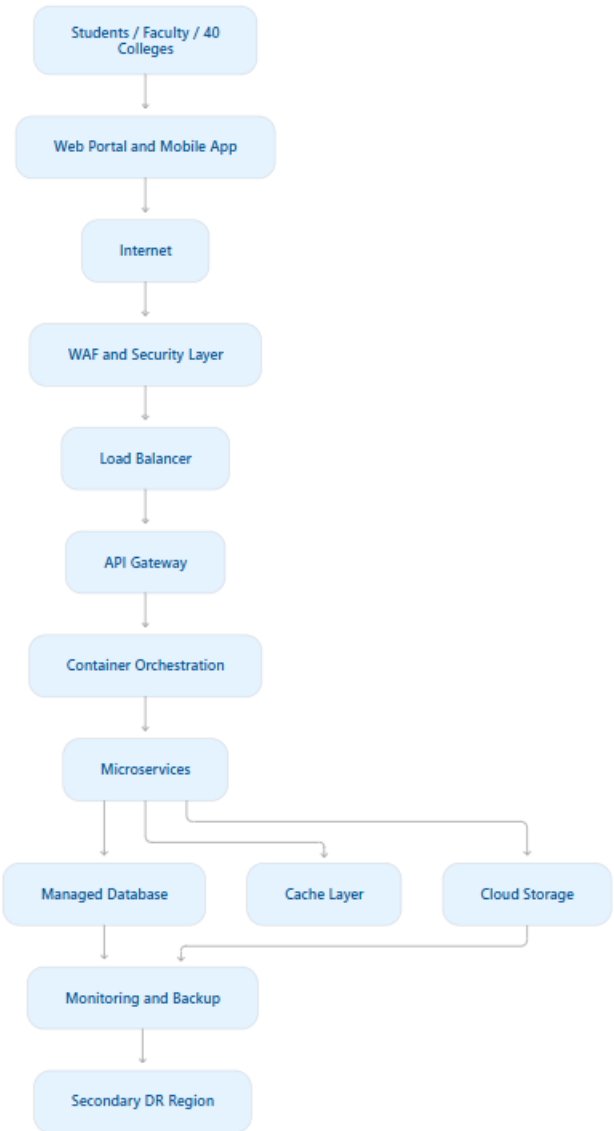
Final Decision: Modify the Original Design

The original hybrid cloud design should be modified rather than completely replaced.

The new design will use API management, container orchestration, serverless services, multi-zone deployment and a secondary disaster recovery region.

Revised Cloud Architecture

Figure 11.1 – Modified Cloud Architecture for 40 Affiliated Colleges



Changes in the Revised Architecture

1. API Gateway

An API gateway is introduced to manage requests from mobile applications and 40 affiliated colleges.

It provides authentication, authorization, routing and rate limiting.

2. Container Orchestration

The application services are deployed using containers managed by a platform such as Kubernetes.

This allows independent services to scale according to demand.

3. Serverless Components

Serverless functions can handle small independent tasks such as notifications and payment-status processing.

This avoids maintaining dedicated servers for every small operation.

4. Managed Database

A managed database service is used to reduce database administration and support automated backup and scaling features.

5. Multi-Zone Deployment

Application instances are distributed across multiple availability zones to reduce the impact of individual infrastructure failures.

6. Disaster Recovery Region

A secondary region stores replicated or recoverable data and supports restoration if the primary region becomes unavailable.

7. Security Layer

A Web Application Firewall (WAF), authentication and tenant-level access controls protect university services.

Revised Service Model Mix

Component	**Service Model**
Legacy SIS	IaaS
Mobile Application APIs	PaaS
Container Platform	Managed PaaS / CaaS
Student Database	DBaaS
Email	SaaS
Notifications	FaaS
Backup and Recovery	DRaaS

Security	Security as a Service

Virtualization Changes

The original plan mainly depended on virtual machines.

The modified architecture uses:

- Virtual machines for legacy applications.
- Containers for scalable application services.
- Serverless functions for event-driven tasks.
- Managed cloud databases for data management.

This approach provides more flexibility than depending only on virtual machines.

11.5 Evidence-Based Defence (2 Marks)

A. Capacity Comparison

Parameter	**Original Design**	**Modified Design**
Users	University users	University users and 40 colleges
Workload	Existing peak	10× existing peak
Computing	Three calculated host equivalents	Approximately 26 host equivalents at the same CPU assumptions
Application Design	Mostly traditional applications	Independently scalable services
Access	Web-based	Web, mobile and external APIs
Availability	Original design	99.9% target
Disaster Recovery	Basic backup approach	Secondary DR region

The 26-host calculation is a capacity estimate, not a fixed purchase recommendation. A managed cloud platform can scale capacity dynamically rather than requiring the university to purchase 26 physical servers.

B. Cost Comparison

The following estimates illustrate the additional operating cost of the modified design.

Expense	**Original PaaS/SaaS Design**	**Modified Design**
Application infrastructure	₹9 lakh	₹13 lakh
Database services	₹4.5 lakh	₹6 lakh
Email and collaboration	₹3 lakh	₹3 lakh
Storage and backup	₹3 lakh	₹4 lakh
Migration and administration	₹4.5 lakh	₹4 lakh

Additional API and security services	Included in general platform assumptions	₹4 lakh
Total for 3 Years	₹24 lakh	₹34 lakh

Additional estimated investment:

\[34-24=10\text{ lakh} \]

The modified solution costs approximately ₹10 lakh more over three years under these illustrative assumptions.

However, it supports a much larger workload, additional college integrations, stronger security and improved availability.

C. Security and Multi-Tenant Isolation

The revised system must implement the following measures:

1. Assign a unique tenant ID to each affiliated college.
2. Implement role-based access control.
3. Use secure authentication for external users.
4. Encrypt student records during storage and transmission.
5. Maintain separate access permissions for each college.
6. Record API activities through audit logs.
7. Apply rate limits to prevent excessive API requests.
8. Verify payment transactions through a trusted payment gateway.

D. Trade-Offs

Advantage	**Trade-Off**
Better scalability	More infrastructure complexity.
Improved availability	Additional redundancy cost.
API integration	Requires API security management.
Container deployment	Requires container administration skills.
Multi-tenancy	Requires careful data isolation.
Multiple cloud services	Potential vendor lock-in.

11.6 Final Decision

The original hybrid cloud solution should be modified to support the university's new requirements.

The modified architecture introduces an API gateway, container orchestration, managed databases, serverless functions, multi-zone deployment and a secondary disaster recovery region.

Although the estimated three-year cost increases from ₹24 lakh to ₹34 lakh, the revised architecture provides the required scalability and supports integration with 40 affiliated colleges.

The university should implement the changes in phases, beginning with API security and mobile application integration, followed by database optimization, container deployment and disaster recovery testing.

Conclusion:

The Day-4 design challenge significantly changes the university's cloud requirements. The original infrastructure cannot independently handle the 10× workload using only three calculated host equivalents. Therefore, modifying the architecture is necessary. The revised solution provides scalable application services, secure multi-tenant access, improved availability and disaster recovery. It offers a practical approach for supporting the university's future expansion while maintaining a controlled operating budget.

MARK DISTRIBUTION

Task	Title	Bloom's Level	Marks
Task 1	Cloud Computing – Introduction & Characteristics	L2 – Understand	3
Task 2	Evolution & Hardware Evolution	L2 – Understand	3
Task 3	Internet Software Evolution	L2 – Understand	3
Task 4	Server Virtualization	L3 – Apply	5
Task 5	Web Services	L3 – Apply	4
Task 6	Service Models: IaaS, PaaS, SaaS	L4 – Analyze	5
Task 7	XaaS	L4 – Analyze	3
Task 8	Design the Cloud Solution Architecture	L6 – Create	5
Task 9	Hands-on Implementation & Documentation	L3 – Apply	4
Task 10	Evaluate & Recommend the Final Solution	L5 – Evaluate	7
	Subtotal		
Task 11	Day-4 Design Challenge	L4 / L5 / L6	8
	TOTAL		50